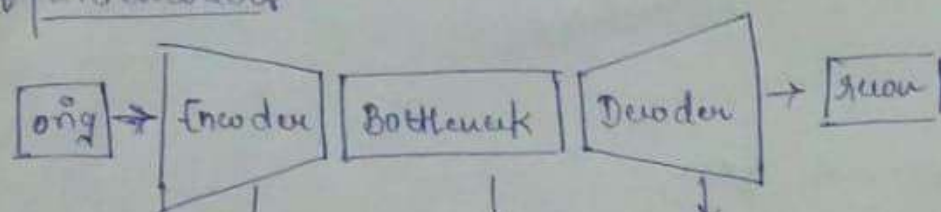


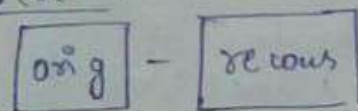
Autoencoders



Effectively compresses input information

Effectively reconstruct input information

Loss:



Explore diff. Loss functions:

- MSE
- Dice
- KL Divergence
- Contrast Loss
- Perceptual Loss
- SSIM
- Tversky Loss
- and much more...

Chrome:
- CNN AE
- AE Appl

* Latency and Latent Dimension

[yt]

- Depla: Autoencoders
Deep learning animated.
5:00 min
7:47 min

Limitations

- Finding Latent space dimension (overlapping clusters)
- Highly dependent on Input data i.e. using techniques like PCA, t-SNE, UMAP.
- Easily affected by Input NOISE $\equiv$ Overfitting

Solution: Regularization

(show 2D linear regression for better understanding)

In case of 2D (Ridge)

$$L(m, b) = \sum_i (y_i - mx_i - b)^2 + (m^2 + b^2) \alpha$$

hyper parameter

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2 + \alpha}$$

$m < \frac{1}{\alpha}$

In case of nD (Ridge)

$$W = (X^T Y) (X^T X + \alpha I)^{-1}$$

decreases m reduces overfitting

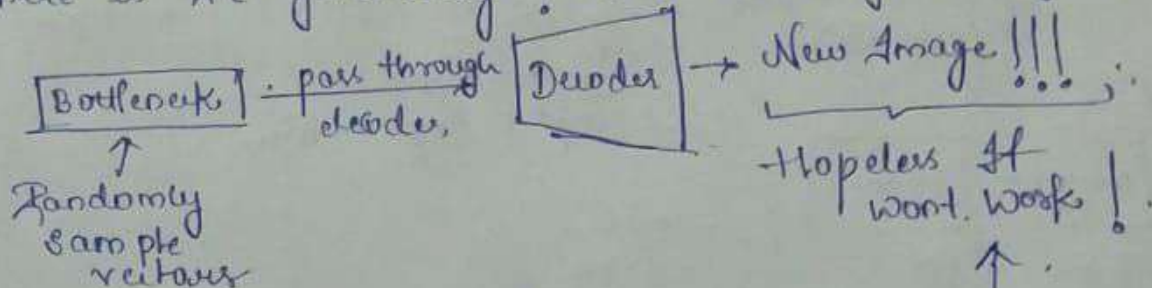
BUT, This is for Linear 'Regression', what about 'Autoencoders'?

Welcome to, Variational Encoders

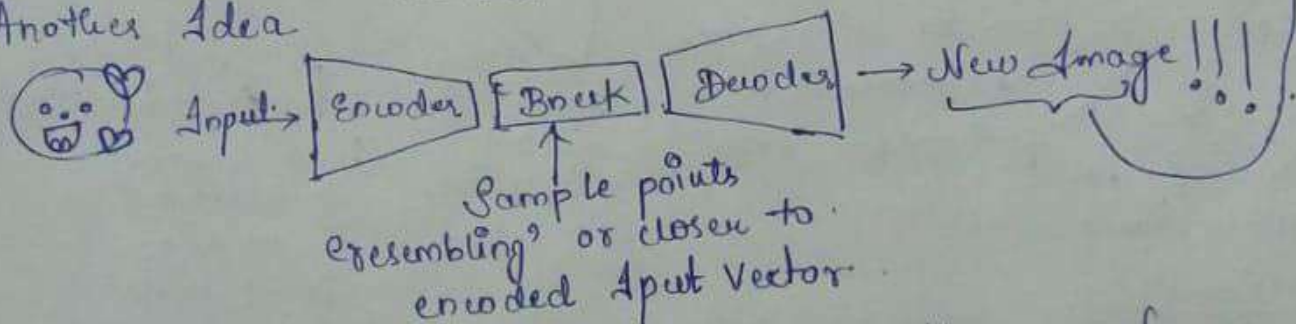
Before going through VAE

Doubt: Parkham explained GAN ✓
Sriram explaining AE ✓

But where is AE 'generating'? what is it generating?



Another Idea



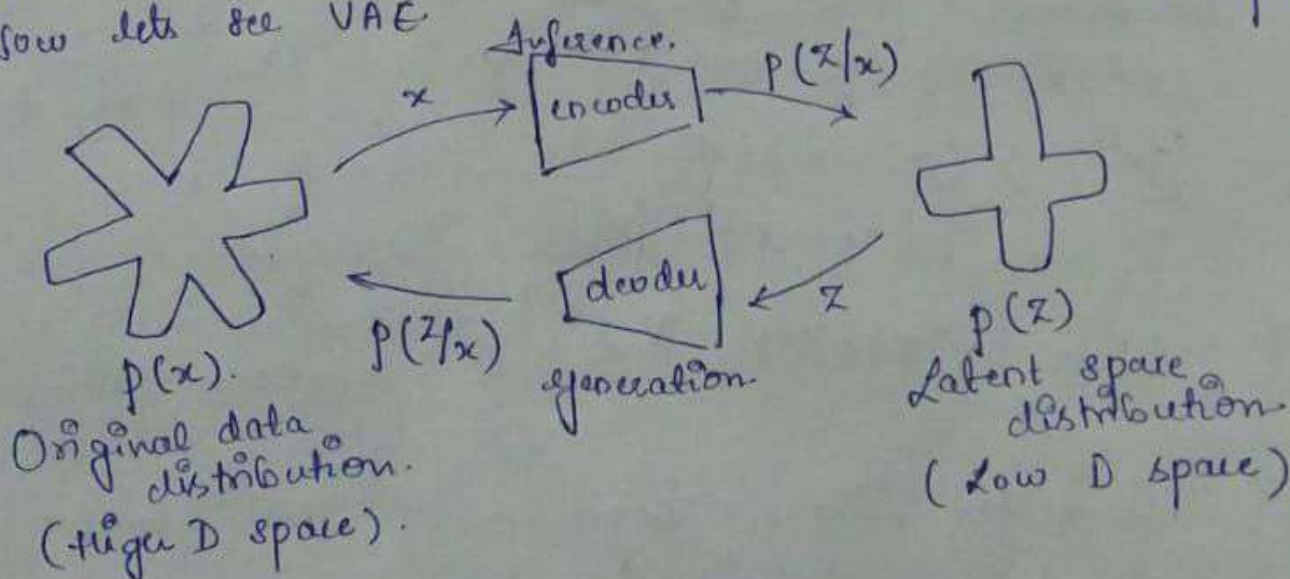
WHY RAM? Heyzaam!! Hey kya Hua Kyu yese hame sataane Lage hoo...

→ Organization of Latent Space.

And dont forget → Overfitting of Input Data. Discontinuity (medium.com).

Paper: Autoencoder Variational Bayes * Diederik P. Kingma.
Father of Adam optimizer.

Now lets see VAE



$p(z/x)$ = Probab that a latent vector z was generated using x

$p(x/z)$ = Probab of reconstructing an image x using z .

Key terminology:

$p(z/x)$ = posterior probability

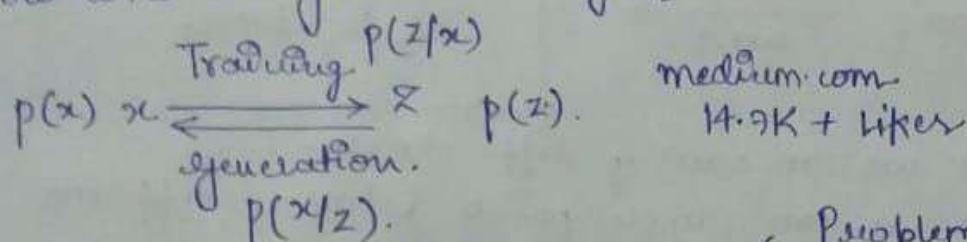
$p(x/z)$ = Likelihood

I Doubt

→ We do not know $p(x)$. We only have samples from $p(x)$

→ We obv do not know how the model will create its latent space $p(z)$.

So how will we generate Image?

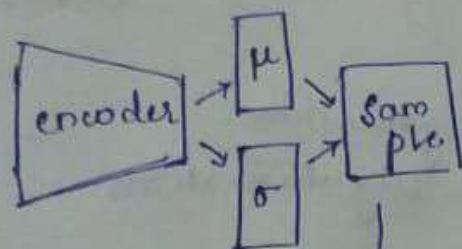


→ Approaching Org. of Latent space. (Problem of continuity).

AE : n -dim Input $\longrightarrow$ less-dim vector (or point).

VAE : n -dim Input $\longrightarrow$ Gaussian distribution (spread of points).

(How? Ans: by determining μ and σ 😊).



So instead of predicting

$z = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$ $\begin{matrix} \rightarrow x\text{-axis} \\ \rightarrow y\text{-axis} \\ \rightarrow z\text{-axis} \end{matrix}$

VAE predicts

$\mu = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$

$\sigma = \begin{bmatrix} 1 \\ 1.2 \\ 2.2 \end{bmatrix}$ $\begin{matrix} \rightarrow N(1, 1) \\ \rightarrow N(2, 1.2) \\ \rightarrow N(3, 2.2) \end{matrix}$

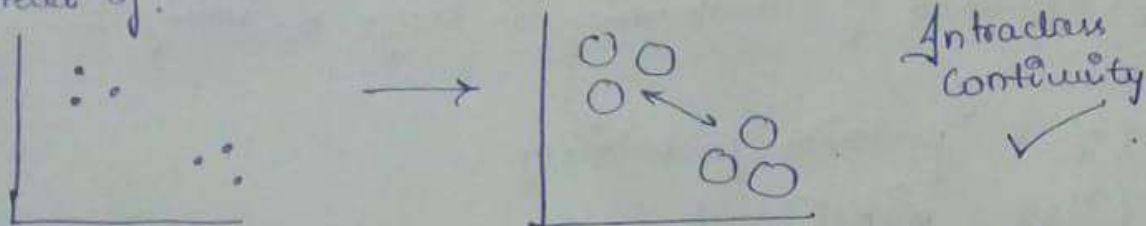
Centre of encoded Input.

Spread / tree of Input class from centre.

Sampling. ★

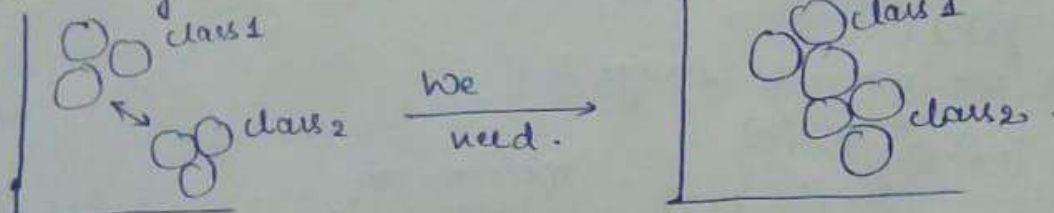
→ Sampling from $z \sim N(\mu, \sigma^2)$ is same as sampling from $X \sim N(0, 1)$ and then scaling $X = \mu + X\sigma$

What problem this step solved?
Instead of.



But continuity problem still exists. How to solve that
(Interclass Continuity)?

Instead of



Why? ans: So that we can overlap diff. classes
So that we can sample points between different
classes as well, but still maintain
class differentiation.

How to solve Interclass Continuity?

→ By restricting μ and σ

How? ans: By using KL-Divergence Loss*

How it works?

yb: Jia-Bin
Huang
(11 min video)

Lets have a Recap, we have seen:

- Working of AE
- Different Loss funcn for AE
- Latent Space
- Limitation of AE → Latent space organization → Overfitting → Regularization
- How to use AE for Image generation

Lets go back to KLD, Very simple!

- MSE: Error between 2 points
- KLD: Error between 2 probability distributions

→ In case of VAE:

KLD is error between $X \sim N(\mu, \sigma^2)$ and $X \sim N(0, 1)$

∴ Now our Loss fu = $\underbrace{\text{Reconstruction error}}_{\text{clustering of similar embeddings}} + \underbrace{\text{KLD error}}_{\text{dense packing of inter clusters}}$

datacamp.
org

for
pytorch
implemen-
tation.

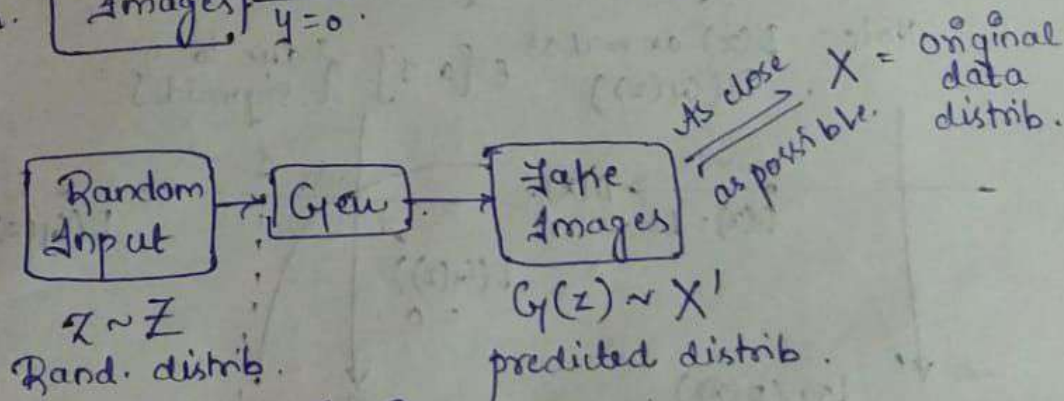
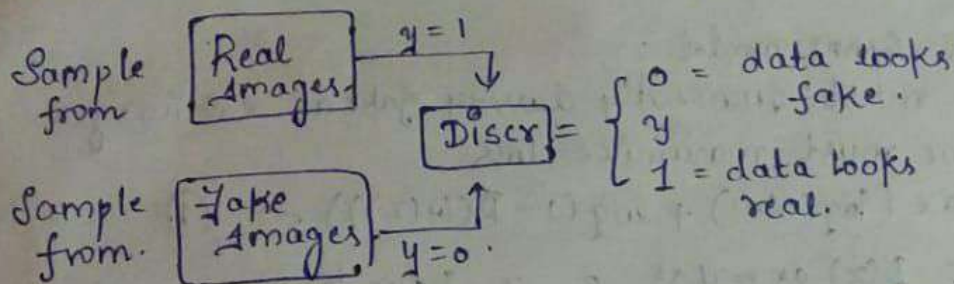
Before moving to CGAN
 Lets recap GAN.

VI* In case of AE or VAE we had a goal
 $\Rightarrow$ Input Image $\rightarrow$ [AE] $\rightarrow$ Reconstructed Output

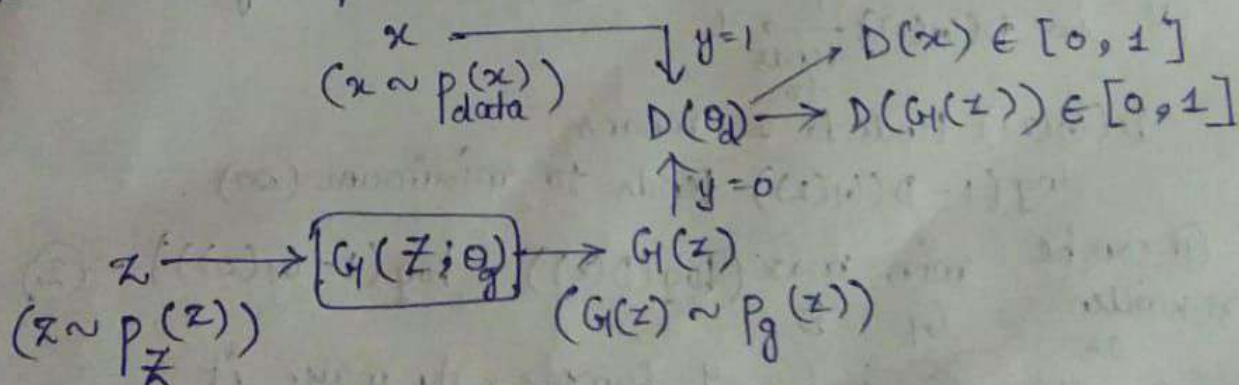
A naive understanding: Output must have some kind of relation (direct) with Input.

In GAN $\Rightarrow$ Lets make model understand distribution of Input(s).

Motivation: Minimax 2 player game.



Mathematical Representation



Our goal: $p_g(z) = p_{data}(x)$, OR
 $x = G(z)$.

Training procedure

1) Discriminator.

* Remember: We are not giving x and $G(z)$ simultaneously

$$BCE(y, \hat{y}) = y \log_2 \hat{y} + (1-y) \log_2 (1-\hat{y})$$

Case 1: when you give Real Image the $\hat{y} = 1$
 Input = x expected output = 1
 $(y = 1)$

$$\begin{aligned} \text{BCE}(\hat{y}, y=1) &= 1 \log D(x) + (1-1) \log_2(1-D(x)) \\ &= \log D(x). \end{aligned}$$

Case 2: when you give Fake Image then $\hat{y} = 0$
 Input = $G(z)$ expected outcome = 0
 $(y = 0)$

$$\begin{aligned} \text{BCE}(\hat{y}, y=0) &= 0 \log D(G(z)) + (1-0) \log(1-D(G(z))) \\ &= \log(1-D(G(z))) \end{aligned}$$

★ Understand.

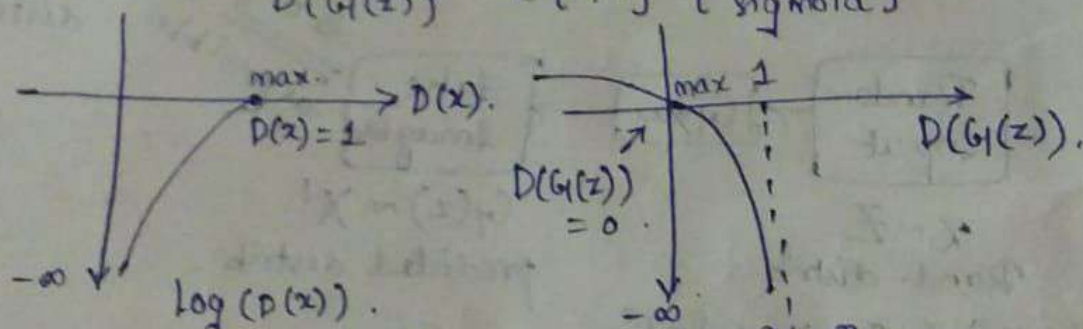
- For a good GAN model:

⇒ Discriminator must incorrectly classify fake as real images

∴ For D we must maximize loss

$$\max (\log(D(x)) + \log(1-D(G(z)))) \quad \text{--- (1)}$$

Note: $D(x)$ as well as $D(G(z)) \in [0, 1]$ } due to sigmoid }



⇒ generator must produce very realistic image

so $D(G(z)) \xrightarrow{\text{tends to}} 1$

$D(G(z))$ tends to 1 means

$\log(1-D(G(z)))$ tends to minimum ($-\infty$).

∴ (1) can be written as $\min_G \max_D (\log(D(x)) + \log(1-D(G(z)))) \quad \text{--- (2)}$

Now this (2) is for 1 sample, to make it for entire dataset we introduce $\sum$ or E summation or expectation.

∴ Final loss function objective.

$$\min_G \max_D V(D, G) = E_{x \sim p(x)} [\log(D(x))] + E_{z \sim p(z)} [\log(1-D(G(z)))]$$

Problems in GAN

- Vanishing gradient
- Mode collapse
- Difficult to achieve Nash equilibrium
- Counting objects in image

Quick Recap

- What is autoencoder?
- Loss functions?
- Limitations
 - Vector space organization
 - Overfitting (Regularizer)
 - ↓ gave rise to
 - Variational autoencoders
 - Annot sampling (2 techniques)
 - Gaussian distributions & Reparametrization
 - Kullback-Divergence → Inter cluster

- Representing GAN
- 2 step training procedure of GAN
- Loss function of GAN
- problems with GAN

How to solve these problems? ↗ also known as Earth Mover distance

1) WGAN (Wasserstein GAN)

- ↳ Vanilla GAN: minimize distance between real and gen images using BCE (or probabilities)
- ↳ WGAN: minimize distance between distributions of real and gen images

* Instead of Disc output $\in [0, 1]$ in WGAN Disc output $\in \mathbb{R} \Rightarrow$ More meaningful feedback to generator.

$\therefore$ This time Disc output $\in \mathbb{R}$, we must also take care of vanishing (& exploding) gradients

* If you notice, instead of discriminating (by giving output as 0 or 1) here the Disc is outputting a number z , hence it is also called 'Critic' and the loss is called 'Critic Loss'.

For the Critic
 $\Rightarrow$ maximize its difference between output on real instances & its output on fake instances

$$\therefore \max(D(x) - D(G(z)))$$

For the Generator

$\Rightarrow$ maximize discriminator's output for its fake instances

$$\therefore \max(D(G(z)))$$

In order to deal with vanishing & exploding gradients it introduces a penalty known as

Lipschitz Continuity Constant (Don't worry).

(No need to remember) penalty = $(\|\nabla_x D(x)\|_2 - 1)^2$
 L_2 norm

2) CGAN (conditional GANs).

$\hookrightarrow$ Vanilla GAN: Input is random noise

$\hookrightarrow$ CGAN: Random noise with labels as input so that there is context based image generation

Let $\mathcal{L}$ be the set of labels given as input along with random noise z and

real image samples x , then loss function =

$$\min_G \max_D \left[\mathbb{E}_{x \sim p(x)_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z|\mathcal{L})))] \right]$$

